

CS223 Computer Architecture & Organization

Instruction Set Architecture



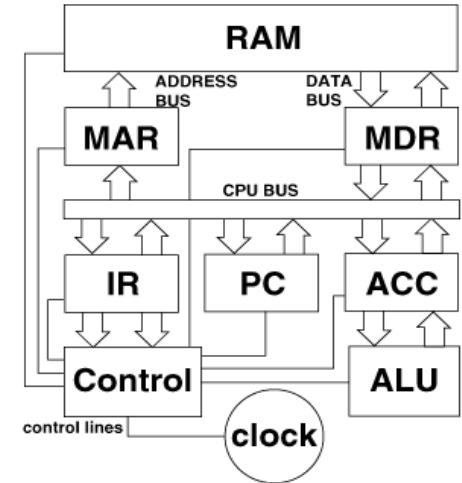
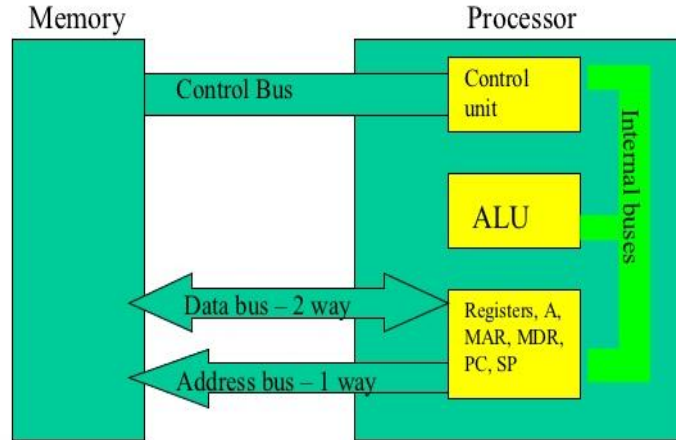
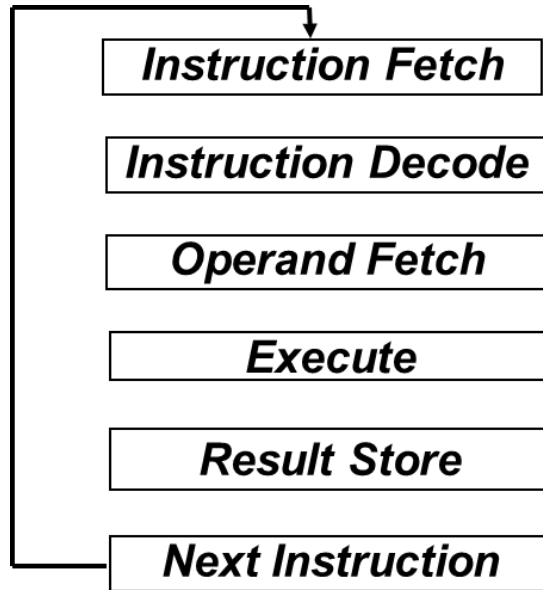
John Jose

Associate Professor

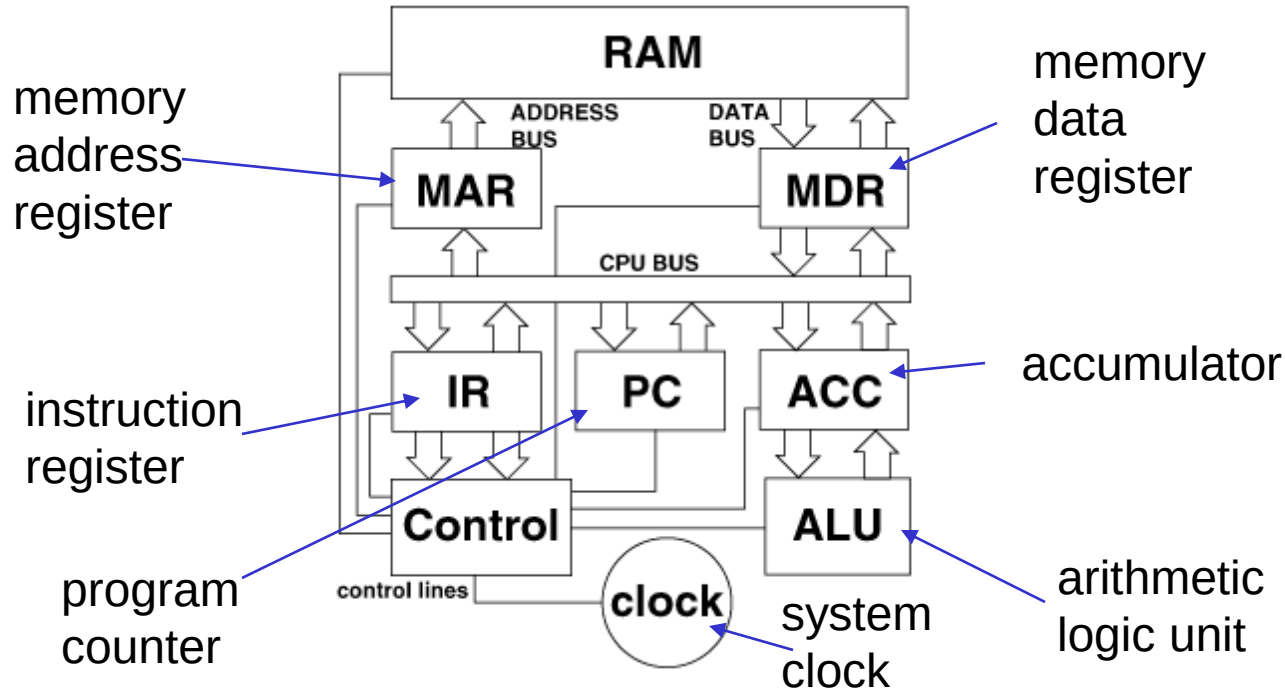
Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Processor Memory Interaction



Inside the CPU



The Instruction

- ❖ An instruction the **most basic unit of command**
- ❖ **Instructions** are words in the language of a computer
 - ❖ **Instruction Set Architecture** (ISA) is the vocabulary
- ❖ The language of the computer can be written as
 - ❖ **Machine language**: Computer-readable representation
 - ❖ **High Level language**: Human-readable representation
 - ❖ **Assembly language**: Human-readable representation

Stored Program & Sequential Execution

- ❖ Instructions and data are stored in memory
 - ❖ Typically the instruction length is the **word length**
- ❖ The processor fetches instructions from memory **sequentially**
 - ❖ Fetches one instruction
 - ❖ Decodes and executes the instruction
 - ❖ Continues with the next instruction
- ❖ The address of the next instruction is stored in the PC.
- ❖ PC is incremented by length of current instruction [Eg: $PC=PC+4$]

Types of Instructions

- ❖ Arithmetic and Logical Instructions
- ❖ Data Movement Instructions
- ❖ Control Flow Instructions

Types of Instructions

- ❖ Arithmetic and Logical Instructions [Execute in ALU]
 - ❖ integer arithmetic
 - ❖ comparing two quantities
 - ❖ shifting, rotating bits in a register
 - ❖ testing, comparing, and converting bits

Types of Instructions

- ❖ Data Movement Instructions [Between Memory Hierarchy]
 - ❖ moving data from memory to the CPU registers [Load]
 - ❖ moving data from CPU registers to memory [Store]
 - ❖ moving data from memory to memory
 - ❖ input and output [I/O]

Types of Instructions

❖ Control Flow Instructions

- ❖ starting a program
- ❖ halting a program
- ❖ skipping to other instructions
- ❖ testing data to decide whether to skip over some instructions

Example Addition Instruction

High-level code

```
a = b + c;
```

Assembly

```
add a, b, c
```

❖ **add**: indicate the operation

❖ **b, c**: source operands

❖ **a**: destination operand

❖ $a \leftarrow b + c$

Variable-Register Mapping

```
b = R1, c = R2, a = R0
```

Assembly Vs Machine Language

```
ADD R0, R1, R2
```

```
1010 0000 0001 0010
```

ISA vs Microarchitecture

❖ ISA

- ❖ Agreed upon interface between software and hardware
- ❖ Compiler writer needs to know ISA generate the target machine code

Program/Application
ISA
Microarchitecture
Circuits

❖ Microarchitecture

- ❖ Specific implementation of an ISA
- ❖ Not visible to the software
- ❖ Microprocessor = ISA, Micro-arch, circuits

ISA vs Microarchitecture

❖ ISA vs Micro-arch

- ❖ ISA is the Interface
- ❖ Micro-arch is the Implementation

❖ Example: Engine acceleration

- ❖ Accelerate pedal (interface)
- ❖ Internal of Engine (implementation)

❖ Add instruction vs Adder implementation

- ❖ Add instruction (interface)
- ❖ Ripple carry / carry lookahead adder (types of implementations)

Program/Application
ISA
Microarchitecture
Circuits

Instruction Set Architecture

- ❖ Instruction: Opcode, Operand

- ❖ Classification of ISA

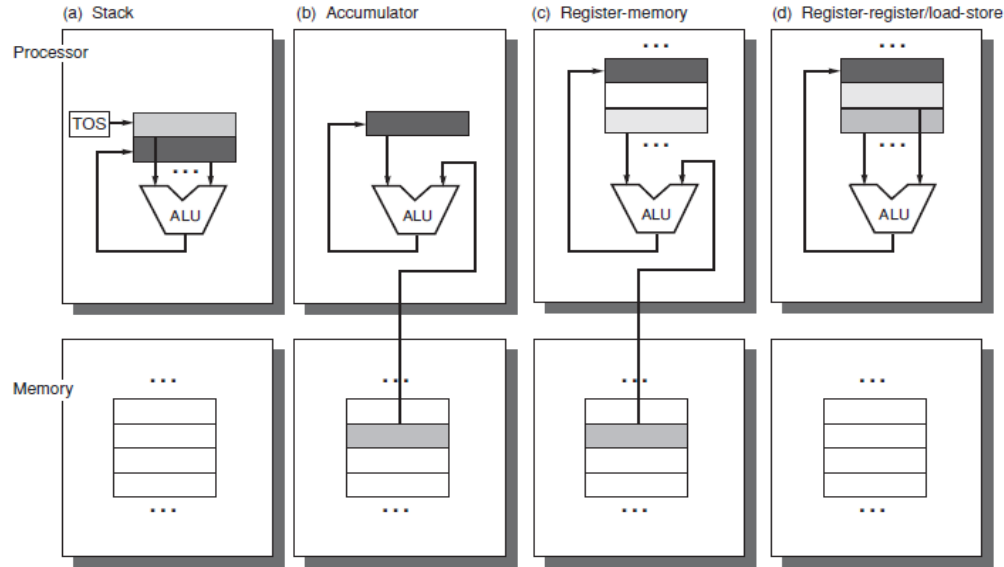
 - ❖ Stack architecture

 - ❖ Accumulator architecture

 - ❖ Register-Memory architecture

 - ❖ Register-Register/Load Store architecture

Instruction Set Architecture



Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

The code sequence for $C = A + B$ for four classes of instruction sets.

Instruction Set Architecture

❖ $A = D * (B + C) - E$

Stack machine

- ❖ Push D
- ❖ Push B
- ❖ Push C
- ❖ Add
- ❖ Mul
- ❖ Push E
- ❖ Sub
- ❖ Pop A

Accumulator machine

- ❖ Load B
- ❖ Add C
- ❖ Mul D
- ❖ Sub E
- ❖ Store A

Load Store machine

- ❖ Load R1, D
- ❖ Load R2, B
- ❖ Load R3, C
- ❖ Add R4, R2, R3
- ❖ Mul R5, R1, R4
- ❖ Load R6, E
- ❖ Sub R7, R5, R6
- ❖ Store R7, A



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>